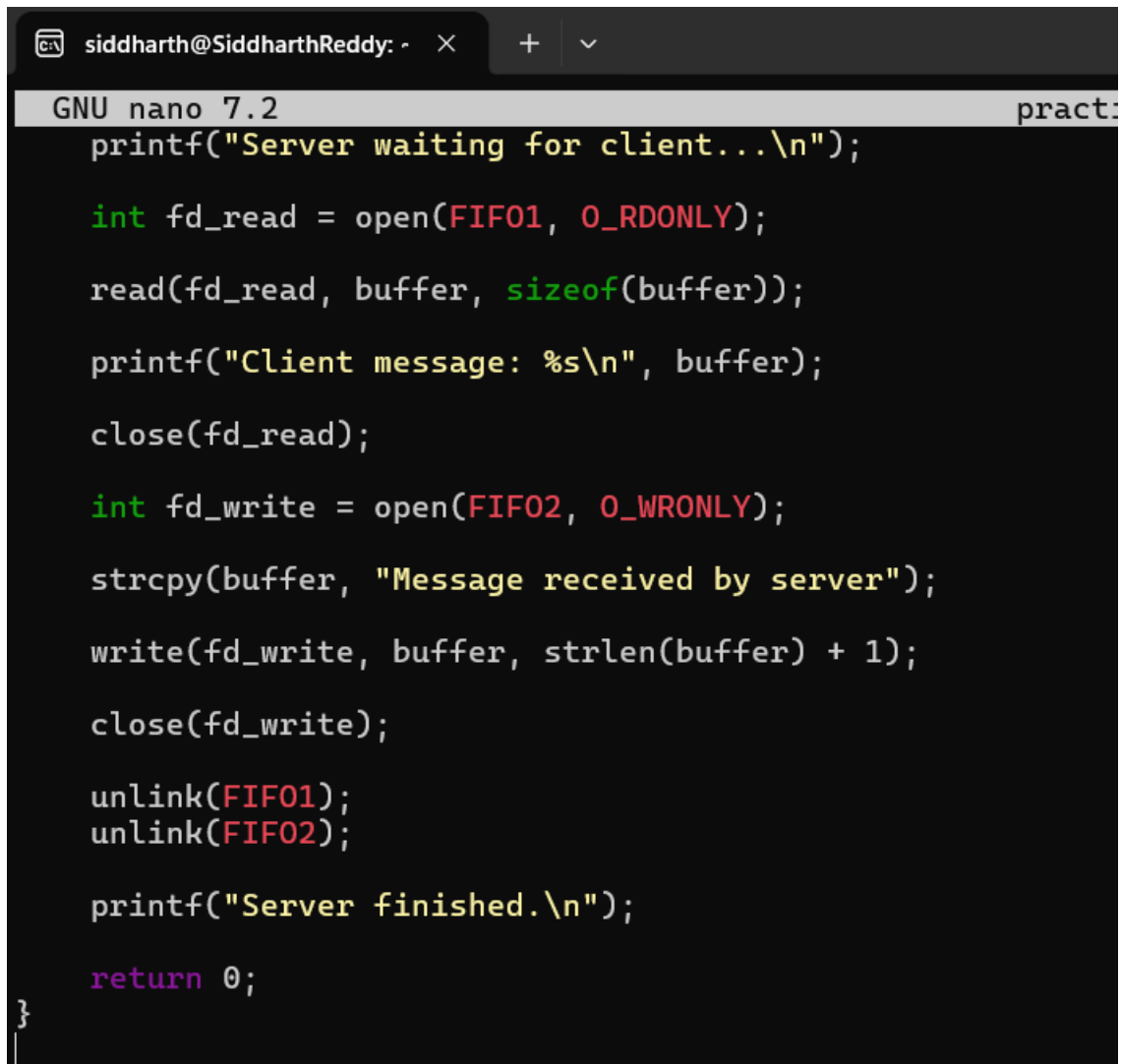


Practical – 6 – OS

```
siddharth@SiddharthReddy: ~  
GNU nano 7.2 pract  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <sys/types.h>  
#include <sys/stat.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
#define FIF01 "client_to_server"  
#define FIF02 "server_to_client"  
  
int main()  
{  
    char buffer[100];  
  
    mkfifo(FIF01, 0666);  
    mkfifo(FIF02, 0666);  
  
    printf("Server waiting for client...\n");  
  
    int fd_read = open(FIF01, O_RDONLY);  
  
    read(fd_read, buffer, sizeof(buffer));  
  
    printf("Client message: %s\n", buffer);  
}
```

Practical – 6 – OS

A screenshot of a terminal window with a dark background. The window title bar shows 'siddharth@SiddharthReddy: ~' and a tab icon. The terminal header indicates 'GNU nano 7.2' and a file named 'pract:'. The code is written in C and uses syntax highlighting: keywords like 'int', 'return', and 'printf' are in green; string literals are in yellow; and file names 'FIFO1' and 'FIFO2' are in red. The code implements a server that reads from a FIFO, prints the message, writes a response to another FIFO, and then unlinks both FIFOs.

```
printf("Server waiting for client...\n");

int fd_read = open(FIFO1, O_RDONLY);

read(fd_read, buffer, sizeof(buffer));

printf("Client message: %s\n", buffer);

close(fd_read);

int fd_write = open(FIFO2, O_WRONLY);

strcpy(buffer, "Message received by server");

write(fd_write, buffer, strlen(buffer) + 1);

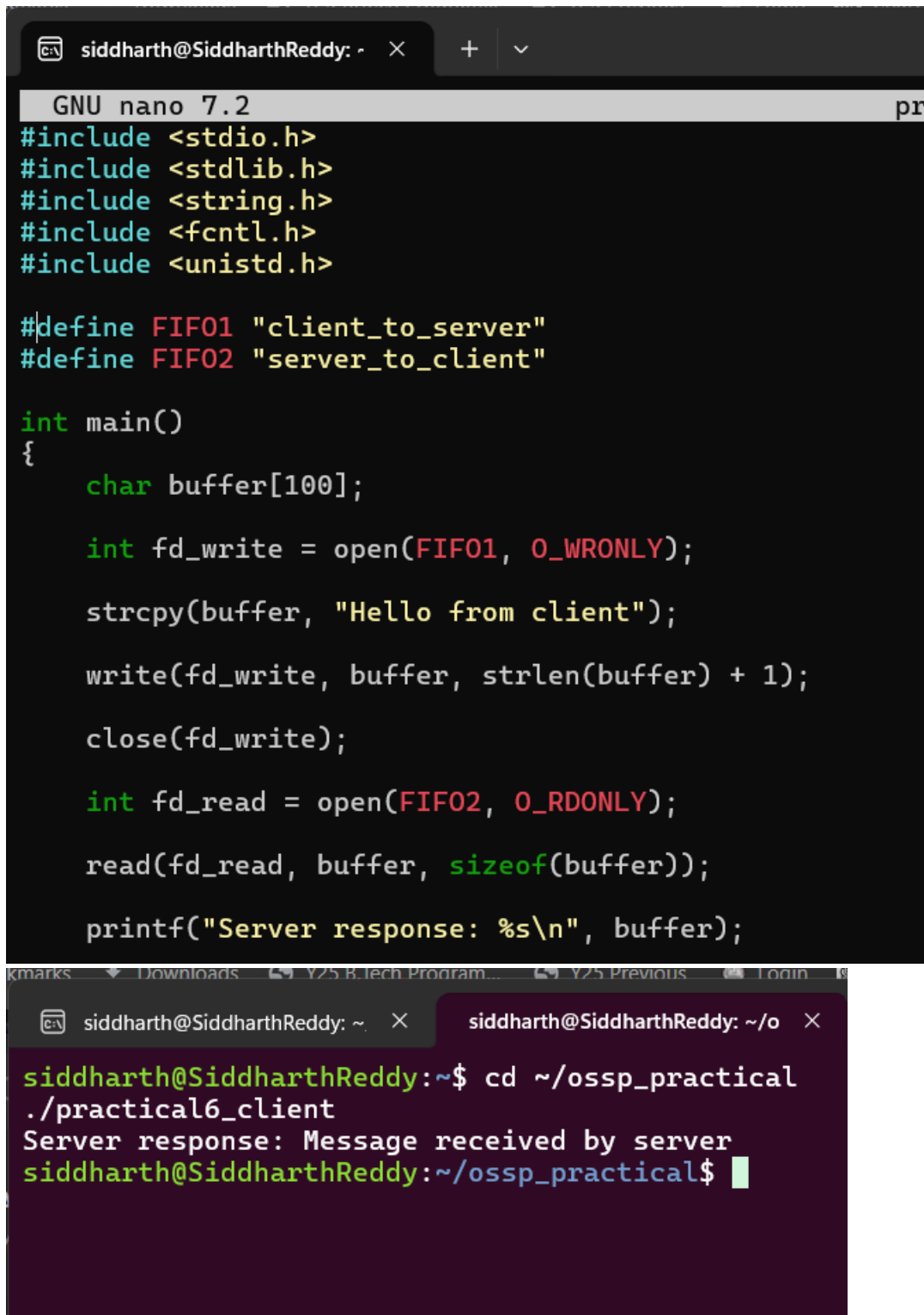
close(fd_write);

unlink(FIFO1);
unlink(FIFO2);

printf("Server finished.\n");

return 0;
}
```

Practical – 6 – OS



The image shows a terminal window with two panes. The top pane is a nano editor editing a C program. The code defines two FIFOs, FIF01 and FIF02. FIF01 is used by the client to write the message "Hello from client". FIF02 is used by the server to read the message. The client program opens FIF01 in write-only mode, writes the message, and closes the file. The server program opens FIF02 in read-only mode, reads the message into a buffer, and prints it. The bottom pane shows the terminal output where the user navigates to the directory ~/ossp_practical, runs ./practical6_client, and sees the output "Server response: Message received by server".

```
siddharth@SiddharthReddy: ~  
GNU nano 7.2  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
#define FIF01 "client_to_server"  
#define FIF02 "server_to_client"  
  
int main()  
{  
    char buffer[100];  
  
    int fd_write = open(FIF01, O_WRONLY);  
    strcpy(buffer, "Hello from client");  
    write(fd_write, buffer, strlen(buffer) + 1);  
    close(fd_write);  
  
    int fd_read = open(FIF02, O_RDONLY);  
    read(fd_read, buffer, sizeof(buffer));  
    printf("Server response: %s\n", buffer);  
}
```

```
siddharth@SiddharthReddy: ~$ cd ~/ossp_practical  
./practical6_client  
Server response: Message received by server  
siddharth@SiddharthReddy:~/ossp_practical$
```

Practical – 6 – OS

```
siddharth@SiddharthReddy: ~$ cd ~/ossp_practical
siddharth@SiddharthReddy:~/ossp_practical$ nano practical6_server.c
siddharth@SiddharthReddy:~/ossp_practical$ nano practical6_client.c
siddharth@SiddharthReddy:~/ossp_practical$ gcc practical6_server.c -o practical6_server
siddharth@SiddharthReddy:~/ossp_practical$ gcc practical6_client.c -o practical6_client
siddharth@SiddharthReddy:~/ossp_practical$ cd ~/ossp_practical
./practical6_server
Server waiting for client...
Client message: Hello from client
Server finished.
siddharth@SiddharthReddy:~/ossp_practical$ |
```

```
GNU nano 7.2 practical6_signal.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nReceived SIGINT (Ctrl+C)\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
    }

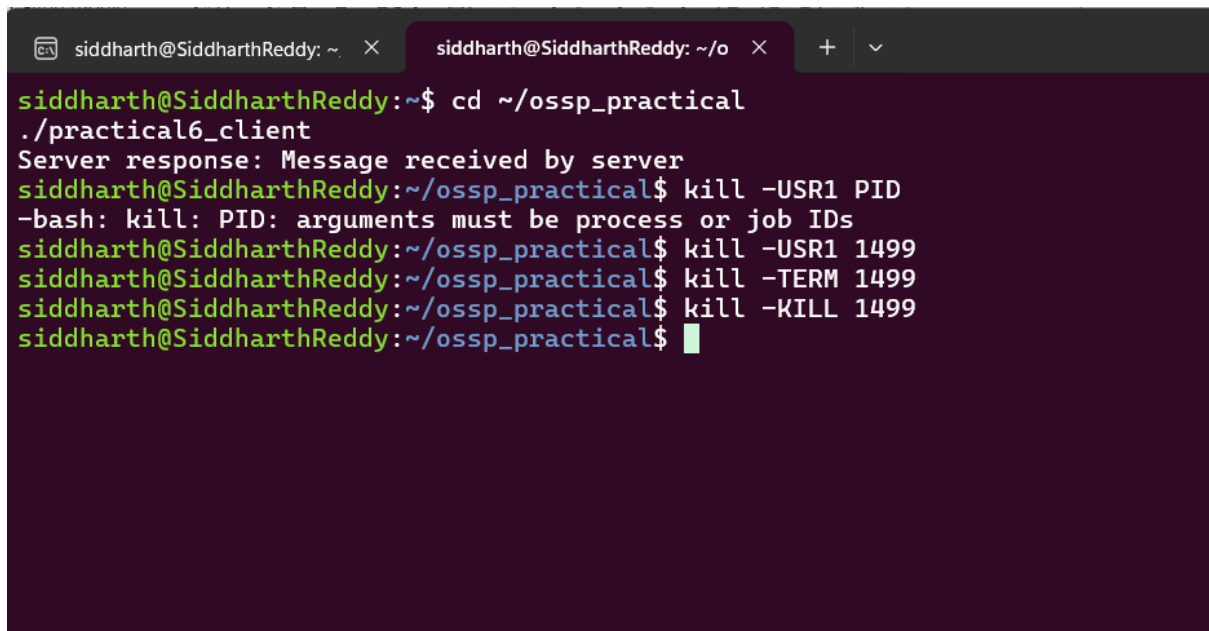
    fflush(stdout);
}

int main()
{
    signal(SIGINT, handle_signal);
```

Practical – 6 – OS

```
siddharth@SiddharthReddy: ~  
GNU nano 7.2 practical6_signal.c *  
    }  
    else if (sig == SIGUSR1)  
    {  
        printf("\nReceived SIGUSR1\n");  
    }  
  
    fflush(stdout);  
}  
  
int main()  
{  
    signal(SIGINT, handle_signal);  
    signal(SIGTERM, handle_signal);  
    signal(SIGUSR1, handle_signal);  
  
    printf("Process ID: %d\n", getpid());  
    printf("Waiting for signals...\n");  
  
    while (1)  
    {  
        pause();  
    }  
  
    return 0;  
}  
|  
  
siddharth@SiddharthReddy:~/ossp_practical$ nano practical6_signal.c  
siddharth@SiddharthReddy:~/ossp_practical$ gcc practical6_signal.c -o practical6_signal  
siddharth@SiddharthReddy:~/ossp_practical$ ./practical6_signal  
Process ID: 1499  
Waiting for signals...  
^C  
Received SIGINT (Ctrl+C)  
  
Received SIGUSR1  
  
Received SIGTERM  
Killed  
siddharth@SiddharthReddy:~/ossp_practical$ |
```

Practical – 6 – OS



```
siddharth@SiddharthReddy: ~$ cd ~/ossp_practical
./practical6_client
Server response: Message received by server
siddharth@SiddharthReddy:~/ossp_practical$ kill -USR1 PID
-bash: kill: PID: arguments must be process or job IDs
siddharth@SiddharthReddy:~/ossp_practical$ kill -USR1 1499
siddharth@SiddharthReddy:~/ossp_practical$ kill -TERM 1499
siddharth@SiddharthReddy:~/ossp_practical$ kill -KILL 1499
siddharth@SiddharthReddy:~/ossp_practical$
```